Deployment & Operations Specification

Sports Venue Booking System — Final Release v1.0

Milestone 5 · Sprint 5 (weeks 13–14) · Team 09 · AI66A

```
unning app:    https://g9-venue-booking-demo.vercel.app
eam:           Team 09 — Sports Venue Booking System (Hanoi pitches/courts)
opic:          Sports Venue Booking System — booking sports venues by time slot

embers:        Nguyen Khac Thu        (11248480)
               Nguyen Khoi Nguyen     (11247331)
               Nguyen Phung Hoa       (11247290)
               Le Quang               (11247343)
               Hoang Thi Ngoc Han     (11247286)

roduct Owner:  @thunopro
crum Master:   @htngochan2802   (Sprint 5 — Sprint 1 @htngochan2802, Sprint 2 @peng543, Sprint 3 @PhunghoaAI, Sprint 4 @lequangk2006-
vs)

epository:     https://github.com/SE-FDA-NEU/G9_AI66A_SE
elease v1.0:   https://github.com/SE-FDA-NEU/G9_AI66A_SE/releases/tag/v1.0
atest green CI run on main:  https://github.com/SE-FDA-NEU/G9_AI66A_SE/actions/runs/41329399157
est accounts:  see the first screen of README.md

ubmitted by:   Nguyen Khac Thu
```

Software Engineering (SE-FDA-NEU) · Milestone 5 Delivery · 06/11/2026

Operations and Maintenance Guide

Operational runbook and release management for the Sports Venue Booking System.

---

1. How to run and release

- **Production URL:** https://g9-venue-booking-demo.vercel.app
- **Health Check:** https://g9-venue-booking-demo.vercel.app/health
- **Hosting Platform:** Vercel Serverless (Python 3.12 runtime) paired with Neon Managed Serverless PostgreSQL and Cloudinary CDN for persistent media.

Release Workflow

A new release reaches production strictly via continuous deployment from `main`: 1. **Branch & PR:** Every change is developed on a dedicated issue branch (`<issue>-<slug>`) and submitted via a Pull Request into `main`. 2. **Automated Verification:** GitHub Actions CI executes automated linting, credential leak scanning, full test suite execution (263+ pytest cases), and deployment readiness checks. 3. **Peer Review:** Base branch rulesets require at least one approving peer review with substantive comments. Self-approvals are strictly blocked. 4. **Merge & Continuous Deployment:**

Once green and approved, the PR is merged into `main`. Vercel automatically detects the green merge commit, builds the serverless package via `api/index.py`, connects to Neon PostgreSQL, and activates the new deployment.

Rollback Procedure

If a production deployment introduces a regression: 1. **Platform Instant Rollback (Fastest, < 30 seconds):** Open the Vercel Dashboard → *Deployments* tab → select the last known green deployment → click the three dots (`...`) → select **Instant Rollback (Promote to Production)**. The platform re-routes production DNS traffic to the previous healthy build immutably without rebuilding. 2. **Git Revert (Permanent trace):** In the git repository, run: `bash git revert -m 1 <merge-commit-hash> git push origin main` CI validates the revert and Vercel automatically deploys the restored version.

---

2. The CI pipeline

The repository CI workflow is defined in `.github/workflows/ci.yml`. It triggers on every push and pull request.

Pipeline Job	What it checks	Execution Time
detect	Checks repository manifest files to detect active runtimes (Python / Node).	~4s
python	Sets up Python 3.12, installs dependencies from `requirements.txt`, runs `ruff` lint check, executes full test suite (`pytest -v --cov`), and publishes JUnit summaries.	~1m 15s
secrets	Scans all git revisions for credentials, API tokens, and private keys. Enforces that no `.env`, `.sql`, or `.sqlite` files are ever committed.	~4s
deployment-readiness	Verifies existence and validity of production manifests (`vercel.json`, `Procfile`, `render.yaml`, `runtime.txt`, `api/index.py`), verifies schema syntax, and validates seed idempotency.	~12s

- **Total CI Run Time:** Approximately 1 minute 20 seconds.
- **Latest Green Run on `main`:** https://github.com/SE-FDA-NEU/G9_AI66A_SE/actions/runs/41329399157

Branch Protection Settings

The `main` branch is protected by organization ruleset `main-protection` (ID: 21296815): - **Pull Request Required:** Direct pushes to `main` are blocked. - **Approving Review:** Requires ≥ 1 approving review from a collaborator with write permissions before merge. - **Block Force Pushing:** Non-fast-forward updates are strictly prohibited. - **Block Branch Deletion:** Deletion of `main` is prevented. - **Bypass Restriction:** `current_user_can_bypass` is set to `never`.

---

3. Monitoring

Application Logs

- **Log Streaming:** The Flask runtime emits structured events to standard output (`stdout` / `stderr`). On Vercel, logs are captured in real time and accessed via **Vercel Dashboard → Project → Logs tab**.
- **Privacy & Security Boundaries:** The application strictly forbids logging user passwords, plain-text credentials, raw email verification tokens, or credit/banking information. Database connection strings are masked.

Health Probes (`/health`)

The public endpoint `/health` validates application health and database reachability: - **Status 200 OK:** Returned when both the web application and PostgreSQL/SQLite database are operational. Body includes `{"status": "ok", "venues": 24}`. - **Status 503 Service Unavailable:** Returned with `{"status": "error"}` if the database connection drops or tables are inaccessible.

Core Metrics to Watch

1. **HTTP Error Rate (5xx percentage):** - *Target:* `< 0.5%` of total requests. - *Detection:* Monitored via Vercel Analytics and runtime log filters (`status:500`). A sudden increase indicates database exhaustion, unhandled exceptions, or schema mismatches.
2. **Response Latency (p95 latency):** - *Target:* `< 500ms` for page renders and `< 150ms` for API endpoints. - *Detection:* Tracked in Vercel Edge performance graphs. If p95 exceeds 1.5s, it indicates slow queries or connection pool starvation.

---

4. Known risks and technical debt

Item	Layer	What could happen	Plan
Serverless Cold Starts	Infrastructure	Free-tier Neon Postgres and serverless lambdas sleep after idle periods, causing the first request to take 3–5 seconds.	Accepted. Course guidelines explicitly permit free-tier sleep delays on first visit.
Concurrent Booking Race Conditions	Data	Two users attempting to book the identical court-hour slot simultaneously could experience a collision.	Closed via database UNIQUE constraint on `(venue_id, date, start_hour)` and transactional rollbacks (#18, #68).
Media Storage Bandwidth Limits	Infrastructure	Excessive high-resolution photo uploads could exhaust Cloudinary	Accepted. App enforces 5 MB upload

Item	Layer	What could happen	Plan
		free-tier bandwidth.	limits and client-side webp compression.
Date-Dependent Seed Bookings	Tests	Hardcoded future dates in test suites could drift over time and cause assertion failures.	Closed via dynamic date offsetting in test fixtures (#88, #92).
Client-Side Translation Flashes	Frontend	Switching between English and Vietnamese may cause a brief flash of untranslated text before DOM mutation finishes.	Accepted. Language selection is cached in `localStorage` and executed synchronously on `DOMContentLoaded`.

---

5. Maintenance plan

Area	What to do	Owner	How often
Bugs & Regressions	Review GitHub Issues, triage error logs, reproduce via automated pytest cases, and ship hotfixes via PR.	@peng543	Weekly
Dependency Updates	Audit `requirements.txt` via `pip audit` and Dependabot alerts for security patches (Flask, psycopg, pytest).	@PhunghoaAI	Bi-weekly
Data Backups & Seed Health	Verify Neon Postgres automated snapshots and ensure local `data/venues.db` seed idempotency remains intact.	@thunopro	Monthly
Production Monitoring	Inspect Vercel runtime logs, verify `/health` uptime, and monitor response latency thresholds.	@lequangk2006-sys	Weekly
Documentation Integrity	Ensure `docs/traceability.md`, `README.md`, and test catalogs remain synchronized with codebase routes.	@htngochan2802	End of every sprint

---

6. Lessons learned

1. **Database Schema Parity Across SQLite and PostgreSQL:** - *What happened:* Initial development relied on local SQLite, but production serverless hosting on Vercel required a hosted PostgreSQL database (Neon). Dialect discrepancies (case-sensitive table names and boolean casting) caused route failures. - *What we changed:* Built `src/db_compat.py` providing a unified query compatibility adapter and dual schemas

(`schema.py` and `pg_schema.py`). - *What we will do differently:* Adopt a dialect-neutral database layer or hosted PostgreSQL instance from Sprint 2 onward.

2. **Date-Dependent Test Flakiness:** - *What happened:* Test fixtures hardcoded specific calendar dates for booking slots. When tests ran near month boundaries, seed collisions created intermittent test failures. - *What we changed:* Replaced static dates with dynamic relative date offsets (`date.today() + timedelta(days=N)`). - *What we will do differently:* Establish strict fixture factories with isolated ephemeral in-memory databases for every test module.

3. **Multi-Subagent Branch Isolation:** - *What happened:* Concurrent feature branches touching shared files (`src/app.py` and `src/templates/base.html`) suffered merge conflicts during sprint wrap-ups. - *What we changed:* Standardized granular route blueprints (`src/routes/`) and isolated CSS components. - *What we will do differently:* Enforce strict modular component boundaries and smaller, continuous daily PR merges.

---

7. Final demo runbook (12 minutes)

Runbook for the Week 15 final demonstration.

Timing	Demo Focus	Action & Screen	Speaker
1 min	Problem & Personas	Introduce amateur sports booking problem in Hanoi; present Customer and Venue Owner personas; list 6 core P0 stories.	Nguyen Khac Thu (@thunopro)
5 min	Live P0 Walkthrough	Open https://g9-venue-booking-demo.vercel.app in an incognito private window. Complete US01–US06 by clicking only. Create booking `BK-000105`, refresh to demonstrate persistence. Intentionally break BR12 (booking past closing) to show user-friendly error handling. Showcase 2 M3 feedback improvements (dual language switcher, responsive sports grid cards).	Nguyen Khoi Nguyen (@peng543) & Hoang Thi Ngoc Han (@htngochan2802)
2 min	Automated Test Suite	Open GitHub Actions test run showing 263+ passed tests. Open `docs/testing.md` at selected P0 story. Introduce intentional boundary failure locally in `src/venues.py`, run pytest to turn test RED, undo to return to GREEN.	Le Quang (@lequangk2006-sys)
2 min	Pipeline & Operations	Display GitHub Actions CI run on `main`, demonstrate `main-protection` branch ruleset, explain instant rollback on Vercel, curl `/health` live (200 OK). Defend PostgreSQL serverless architectural decision.	Nguyen Phung Hoa (@PhunghoaAI)
2 min	Lessons & Technical	Present one key lesson learned (SQLite-Postgres compatibility) and explain cold-start technical debt mitigation plan. Address jury	Full Team

Timing	Demo Focus	Action & Screen	Speaker
	Debt	questions.	